

RS Managed Agents — Plugin Taxonomy & Setup Guide

Last updated 28 September 2026

One repo, one marketplace, plugins named for the job. Everyone with repo access sees everything. Access control is deferred — why, and what would trigger it, is at the end.

Strategy

One rule decides what goes in a plugin: **would one person install these together, for one kind of work?**

We got it wrong once. The repo started as a doc-generation workshop, so the plugin holding everything was called `rightshift-doc-gen`. It then picked up a Playwright test runner, a diagram renderer and a seven-hour training course. Three of ten skills generate no documents.

The cause was structural. `.claude-plugin/plugin.json` sat at the repo root with `source: "./"`, auto-discovering everything in `skills/`. Anything that landed there joined that plugin.

What follows from the rule:

- **Business line is not a boundary.** Accelerator and Studio product people want the same skills. Splitting them means maintaining one skill twice.
- **Topic is not a boundary either.** The person writing a test plan and the person running Playwright against the build are the same person that afternoon. The person pricing a deal is neither.
- **A plugin that needs explaining is wrong.** "The course is in doc-gen for historical reasons" is the tell.

The taxonomy

Twelve plugins, all prefixed `rs-`, named for the job not the department. The middle column is what a skill router matches against — when to reach for it, and what it is not for.

Plugin	Reach for it when	Skills
`rs-research`	You need to understand a market, a prospect company or a competitor before anything is scoped; sharpen discovery questions before a client call; or turn a vague client complaint into a sharp problem statement. Ends where client-facing documents begin.	company-research, market-analyzer, competitive-and-market-research, customer-discovery-question-optimizer, problem-statement-analyzer
`rs-scoping`	You have notes, transcripts or emails from a client and need a business case, then a product spec, then an architecture and component inventory, then a man-day estimate priced against the rate card. The chain that ends in a number. Not for presenting or selling that number.	opportunity-packet, brd-generator, prd-generator, technical-design-generator, estimation-generator, google-estimation-sheet-extractor, feature-pov-narrative
`rs-sales`	The scope exists and now has to be won: proposals and platform decks, startup pitch decks, contracts, outbound campaigns, and review of anything client-facing before it goes out. Not for producing the scope itself.	gamma-prototype-proposal-deck, gamma-platform-proposal, startup-pitch-deck-generator, contract-creator, outbound-sales-machine, pitch-deck-analyzer, executive-summary-evaluator
`rs-content`	You are writing something published under the RightShift name for an audience rather than a client: LinkedIn posts, blog articles, newsletters, case studies, speaking-engagement campaigns, one-pagers. Also the voice guide	deep-voice-guide, linkedin-post-machine, **blog-agent** *(planned)*, case-study-and-testimonial-generator, rightshift-traction, ai-adoption-one-pager, speaking-engagement (pitch deck, pre-event, post-event, RSVP

Plugin	Reach for it when	Skills
	everything here is written against. Not for client deliverables.	enrichment), cross-page-editorial-consistency-audit, client-communication-qa
`rs-design`	You are producing interface or visual design — SaaS UI in Figma, a design reference converted to a CSS spec — or evaluating a page, a site or a Figma file against the brief it was meant to satisfy. Also brand identity. Not for building it in code.	saas-uiux-designer-figma, design-reference-to-css-spec, figma-page-review-vs-brief, page-design-evaluation-generic, rightshift-site-design-evaluation, rightshift-identity
`rs-prototype`	You need something running or clickable fast — an HTML or Next.js prototype, a Spark or Propel engagement — or you have a prototype and need to decide which features become the MVP. Not for production delivery.	html-prototype-generator, nextjs-prototype-generator, rightshift-spark, rightshift-propel, rightshift-feature-generator, prototype-to-mvp-feature-generation, prototype-feature-audit-and-ideation
`rs-delivery`	A PRD is approved and the work has to ship: breaking features into per-feature specs a coding agent can execute, then implementing, reviewing and shipping as a PR. Not for prototypes, not for deciding what to build.	delivery-agent, spec-generator
`rs-qa`	You need a risk-based test strategy and coverage matrix, executable test cases with Gherkin, or to run those cases against a live application through Playwright and record real verdicts. Not for code review.	test-planner, test-case-generator, test-executor
`rs-project-management`	The work is sold and now has to be run: mapping a PRD into sprints and tickets, tracking delivery against plan, weekly client status reports, reconciling logged hours	prd-to-linear-sprint-mapper, hour-reporting-invoice-reconciliation *(idea stage)*, **weekly-status-report** *(planned)*, **delivery-variance-tracker** *(planned)*

Plugin	Reach for it when	Skills
	to invoices. Not for deciding what to build or writing code.	
`rs-product`	A team lead is learning — or running — the course that trains turning a client's stated want into a spec a designer and engineer can build from. Training, not production.	product-thinking-course
`rs-chatbot`	You are building a knowledge-grounded chatbot for a client or internally, defining its personality, or specifying how it behaves when it does not know.	chatbot-forge, chatbot-personality-rulebook, chatbot-behaviour-and-fallback
`rs-common`	Shared utilities any other plugin may lean on: rendering a D2 diagram, applying the document formatting standard. Install alongside whatever else you have.	d2-diagrams, document-formatting-standard

Bold entries don't exist yet. `rs-project-management` starts thin — one working skill, one at idea stage. It exists so the next PM skill has a home.

Two naming rules: always the `rs-` prefix, and the name says the job.

Repository structure

`IntuitioLabsLLC/rightshift-managed-agents` is the marketplace. Plugins are authored in `src/plugins/` and generated into `plugins/` by `make sync`. Never hand-edit the generated directory.

```
rightshift-managed-agents/
  .claude-plugin/
    marketplace.json          # the catalogue - one entry per
plugin
                              # NOTE: plugin.json is DELETED
from here
  plugins/                    # GENERATED by 'make sync' - ne
```

```
ver hand-edit
    rs-scoping/
      .claude-plugin/plugin.json
      skills/
        brd-generator/SKILL.md
        prd-generator/SKILL.md
    rs-content/
    rs-project-management/
  src/
    plugins/                    # SOURCE OF TRUTH - edit here
      rs-scoping/
        .claude-plugin/plugin.json
        skills/<skill>/SKILL.md
        prompts/                # versioned prompt sources for
THIS plugin
          01-brd-generator/{current,versions,improvements}/
          02-prd-generator/{current,versions,improvements}/
      rs-content/
    standards/                  # shared across plugins
    tools/verify_skills.py
  docs/
  Makefile                      # ONE plugin list
```

Three things matter here:

- **Root is no longer a plugin.** Deleting `.claude-plugin/plugin.json` from the root removes the catch-all. A new skill has to be placed in a named plugin.
- **Numbered prompt folders move inside their plugin.** `src/01-` through `src/08-` serve one plugin. The `current` / `versions` / `improvements` convention stays as it is.
- **The Makefile collapses to one list.** Four categories today — `SKILLS`, `UTIL_SKILLS`, `PM_SKILLS`, `PLUGINS` — each with its own sync, dist and install lines. After this, a new plugin costs one entry.

```
PLUGINS := rs-research rs-scoping rs-sales rs-content rs-desi
gn \
```

```
            rs-prototype rs-delivery rs-qa rs-project-manageme
nt \
            rs-product rs-chatbot rs-common
```

Each plugin's `plugin.json` carries its own name, description and version. Skills are auto-discovered from the plugin's `skills/` directory, so adding a skill means adding a directory.

Setup tasks

On a branch, in order. Steps 1–6 are mechanical. Step 7 reaches users — don't run it until the rest verifies clean.

1. **Move each plugin's source into `src/plugins/`.** Mapping is the taxonomy table. The ~40 prompts under `src/prompts/<category>/` move into the owning plugin's `prompts/`. Keep `current` / `versions` / `improvements` intact. *Done when:* `src/prompts/` is empty.
2. **Give each plugin a `.claude-plugin/plugin.json`.** Description comes from the taxonomy table. Copy the shape of `src/chatbot-forge/.claude-plugin/plugin.json`. *Done when:* twelve manifests, all valid JSON.
3. **Delete `.claude-plugin/plugin.json` from the repo root.** Leave `marketplace.json` there. *Done when:* root has a marketplace manifest and no plugin manifest.
4. **Rewrite the Makefile around one `PLUGINS` list.** Drop `SKILLS`, `UTIL_SKILLS`, `PM_SKILLS` and their loops. `make sync` copies `src/plugins/<name>/` to `plugins/<name>/`. The `*_SRC` prompt copying becomes internal to `rs-scoping` and `rs-qa`. *Done when:* `make sync` regenerates `plugins/` and `git status` is clean after.
5. **Update `src/tools/verify_skills.py`.** It walks a flat `skills/` today. Make it walk each plugin's `skills/`, and check every plugin has a manifest. *Done when:* `make verify` passes, and fails if you delete a `SKILL.md`.
6. **Rewrite `marketplace.json`.** One entry per plugin, `source` is `./plugins/<name>`. Keep the `superpowers` entry. Check the `renames` field — it exists for migrations and may map `rightshift-doc-gen` to its successors, saving a manual reinstall. Confirm the semantics first. *Done when:* twelve RightShift plugins listed, plus superpowers.

7. **Release and tell the team.** Conventional Commits drive `standard-version`, which bumps `package.json`, every `plugin.json` and `marketplace.json`. Breaking change — `rightshift-doc-gen` stops existing — so major version. *Done when:* tagged and pushed, and everyone has run `/plugin marketplace update rightshift` and reinstalled.

What breaks

Renaming plugins invalidates existing installs. Anyone on `rightshift-doc-gen` uninstalls and installs the successors. One-time cost, small team, cheaper now than later.

Updating a skill

The rule that catches people out: **an update ships the whole plugin, never one skill, and nothing reaches anyone unless the version number changes.** Editing and pushing is not enough.

1. **Edit under `src/plugins/<plugin>/`.** Never in `plugins/` — `make sync` overwrites it. For prompt-backed skills, promote into `prompts/<stage>/current/`; the old version stays in `versions/`.
2. **`make verify`.** Catches a missing name or description, a bad manifest, an empty plugin.
3. **`make sync`.** Refuses to run if `plugins/` has uncommitted edits — that's the guard against hand-editing.
4. **Commit with a Conventional Commit type.** `prompt:` for a prompt or skill change, `feat:` for a new skill or plugin, `fix:` for a correction. The type drives the version bump and the changelog.
5. **Release.** `standard-version` bumps `package.json`, every `plugin.json` and `marketplace.json`. Skipping this is how a change fails to reach anyone.
6. **Push, with tags.**

How the update reaches people

Automatic, where the marketplace has auto-update on: after the user's first message in a session, Claude Code waits up to ten minutes, refreshes and

updates the plugin on disk. The running session keeps the old version and shows `Plugin updated: <name> · Run /reload-plugins to apply`. Loads on next launch either way.

Manual:

Where	Command
In a session	`/plugin marketplace update rightshift`
In a session, one plugin	`/plugin install <plugin>@rightshift --upgrade`
From a shell	`claude plugin update <plugin>@rightshift`

The version pin

Each `plugin.json` pins an explicit `version`, so users get nothing until that string changes. Omitting `version` would make Claude Code derive it from the commit SHA, and every push would propagate.

Keep the pin — deliberate releases, meaningful changelog. It makes step 5 mandatory.

> **Live example.** `product-thinking-course` was added on 28 September with the version left at 2.4.0. Nobody who already had `rightshift-doc-gen` installed received it.

Checking the catalogue

Question	How to answer it
What plugins exist?	`/plugin` in a session, or read `.claude-plugin/marketplace.json`
What skills are in a plugin?	List `plugins/<name>/skills/` — skills are auto-discovered from there
What do I have installed?	`/plugin` shows installed plugins and their state
What version am I running?	`claude plugin list`, or the `version` in the installed `plugin.json`
What changed between versions?	`CHANGELOG.md` at the repo root, generated from Conventional Commits
Is my copy current?	`/plugin marketplace update rightshift`

First install:

```
/plugin marketplace add IntuitioLabsLLC/rightshift-managed-ag
ents
/plugin install rs-content@rightshift
```

The repo is private, and marketplace access is repo access — sort GitHub access before someone's first day.

Future: splitting by role

Not now. Here's why, and what would change it.

Why deferred. A marketplace has no per-plugin permissions. Entries carry no audience field, and everyone who adds a marketplace sees the whole catalogue — the Claude Code docs say separate audiences need separate marketplaces. Nobody here is harmed by seeing a plugin they don't use.

What would trigger it:

- A contractor needs `rs-content` but shouldn't read the rate card in `rs-scoping` or the contracts in `rs-sales`
- A client-embedded person needs delivery plugins but not commercial ones
- Headcount grows past the point where "everyone is trusted with everything" was actually decided

How it would work. The monorepo stops being the marketplace and becomes source of truth. A publish step copies each plugin into audience repos, each with its own `marketplace.json`. GitHub teams grant read per repo.

```
rightshift-managed-agents          source of truth, maintaine
rs only
  |
  +- make publish AUDIENCE=content
       |
       +- rs-agents-content      rs-content, rs-common
       +- rs-agents-delivery     rs-delivery, rs-qa, rs-proto
type, rs-common
```

```
        +- rs-agents-commercial   rs-scoping, rs-sales, rs-res
earch
```

The option we're skipping. Managed settings can set `enabledPlugins` to `false`, which hides a plugin and blocks installing it, and users can't override it. Pushed per device group via MDM, different groups get different catalogues. Two problems: server-managed settings from the claude.ai admin console apply org-wide and can't target subgroups, so it needs real MDM; and it blocks installing, not reading — anyone who can clone the repo still reads every prompt. For contractors across Atlanta, India and Ukraine, repo segmentation is cheaper and actually protects something.

Design for it now, for free:

- Keep each plugin self-contained — no cross-plugin file references
- Concentrate anything confidential in as few plugins as possible. Today: rate card in `rs-scoping`, contracts in `rs-sales`, voice guide in `rs-content`